HANDBOOK

# Workstreams

## Parallel, disposable worktrees with a durable ledger

> Spin up isolated worktrees for every unit of work, throw them away when they merge, and keep one ledger that always knows where each branch stands.

---

Version 0.40.1 · September 2026

## CONTENTS

# The idea

A workstream is one feature split into many small units of work, each living in its own throwaway git worktree — plus optional **spikes** for store-only research when you need to investigate before you build.

When a feature is too big for a single branch, you break it into **units** — a slice, a follow-up, a stacked layer. Each unit gets an isolated git worktree and branch of its own, so units never step on each other and can be built, reviewed, and merged independently. The worktree is a pure code checkout: a scratch space you can create, abandon, and recreate at will.

Some phases don't fit that model — an audit, an exploration, resolving open questions before a unit starts. Those are **spikes**: durable store entries with tasks and artifacts, no worktree and no PR. They block units (and other spikes) via the same `needs=` graph, and they always end by amending the umbrella design spec.

The catch is that a checkout can vanish — dropped, garbage-collected, or replaced — so nothing you care about keeping can live inside it. Every durable datum — a unit's charter of intent, its progress and follow-ups, its history of decisions and restacks, and its identity map from unit to repo and branch — lives in a single **global store** under `~/.local/share/workstreams/`, one directory per workstream, shared across repos. Spike research lives there too (`spikes/`, `artifacts/`). Volatile facts that git and GitHub already own (current branch, base, PR number, status) are never copied in; they are derived live.

```
~/.local/share/workstreams/<ws-id>/
  workstream.md          # metadata only
  units.md               # append-only ledger (unit ↔ repo/branch identity map)
  spikes.md              # append-only ledger (spike identity — no branch=)
  backlog.md             # planned units + deferred follow-ups
  focus.md               # outcome queue (steers ws-next proposals)
  units/<unit-id>/
    charter.md           # STATIC: why this unit exists — the north star
    progress.md          # MUTABLE current state: Tasks + Follow-ups checklists
    log.md               # APPEND-ONLY: created, dropped, restack, decision, note
    prewalk.md           # optional: read-only exploration digest (superpowers-prewalk)
    critic.md            # optional: post-complete review (review/ws-critic)
  spikes/<slug>/
    charter.md           # STATIC: what to investigate
    progress.md          # MUTABLE: Tasks + Needs only (no Follow-ups)
    log.md               # APPEND-ONLY: created, dropped, decision, plan
    artifacts/           # working drafts, spec-before snapshots
```

> **Golden rule.** The worktree holds only code. Drop and recreate it freely — all progress, history, and identity survive in the store.

## The workflow at a glance

brainstorm · spec · decide the splits → `ws-init` → `ws-focus` →

REPEAT UNTIL SHIPPED

`ws-start` → `ws-resume` → `ws-next` ↻

`ws-board` — glance at status anytime, from any session; read-only. Spikes appear inline with a `[spike]` tag on the same kanban columns.

**Research before build?** Run `ws-spike` when an audit or investigation must finish (and amend the design spec) before a unit starts — then `ws-block` the unit on the spike if needed.

**You never decide the whole split up front.** `ws-init` creates an empty container — it never reads the design to invent units. Start the next unit with `ws-start` whenever you know what it is, and when you don't, `ws-next` proposes one: it reads the design (and any accumulated follow-ups) once nothing else can move, and you pick. Recording a deliberate deviation from the design is `ws-backlog --plan`.

# Installation

One command installs all the skills — through the Agent Skills CLI or the Claude Code plugin marketplace; the store and its config scaffold themselves on first use.

Workstreams ships as a set of `ws-*` skills in the portable **Agent Skills** format: the workflow commands, the shared contract (`ws`), and this guide (`ws-guide`). Install once and the `/ws-*` commands are available in every repo, because the store is global rather than per-project.

### Prerequisites

- **git** with worktree support (2.5+) — every unit is a separate `git worktree`.
- **GitHub CLI** (`gh`), authenticated with `gh auth login` — the default `forge` flavor derives PR state and base from it.
- An agent that supports **Agent Skills** — Claude Code, Cursor, Codex, and others.

### Option A — Agent Skills CLI (skills.sh)

Works with any agent on the Agent Skills standard — Claude Code, Cursor, Codex, and 20+ others. Installs all the skills and keeps them updatable.

```
npx skills add caioariede/claude-plugins
npx skills update    # refresh installed skills later
```

### Option B — Claude Code plugin marketplace

Claude Code only. Point it at this repo's marketplace and install the plugin.

```
/plugin marketplace add caioariede/claude-plugins
/plugin install workstreams@caioariede
```

**Pick one channel.** Installing through both registers the skills twice in Claude Code. Either way, verify with a bare `/ws-config` — it confirms the skills resolve and checks your tools, flagging a missing `gh` before your first `ws-init` fails.

**State scaffolds itself.** The global store at `~/.local/share/workstreams/` and its `flavors.ini` are created on the first `ws-init` — nothing to set up by hand.

### First run

Run `/ws-config` once after installing. It detects which tools you have — `wmx`, superpowers, `gh` — and offers to activate the matching flavors for any group you haven't set; accept and you're configured. It also flags an active flavor whose tool is missing. Defaults are `git-worktree` for worktree management, `gh` for the forge, and `none` for spec-driven development; change any of them later with `ws-config set` — see the Flavors section.

```
$ /ws-config
$ /ws-config set spec-driven-development superpowers
$ /ws-config set spec-driven-development superpowers-prewalk
$ /ws-config set-config agent cursor
$ /ws-config set-config cheap-model.cursor <your-cheap-model>
$ /ws-config set-config frontier-model.cursor <your-frontier-model>
```

# The workflow commands

The workflow commands cover the full life of a unit of work — from spinning up a container to tearing a branch down, plus deciding what's next and configuring flavors — while every durable fact lives in the global store, never in a worktree.

> **Golden rule.** Worktrees are disposable code checkouts; all durable state lives in the store. Every command loads the shared contract (the `ws` skill) before it acts, and never stores what git or GitHub already owns (branch, base, PR number, status — all derived live).

## `ws-init` `<workstream name>`

**Use when** you need a new workstream and none exists yet — before starting any unit with `ws-start`.

Computes the `ws-id` (`<YYYY-MM-DD>-<slug>`) and creates the store container: `workstream.md` metadata, an empty append-only `units.md` ledger, the `units/` directory, empty `spikes.md` and `spikes/`, an empty `backlog.md`, and an empty `focus.md`. Sets up the container only — it creates no unit, no spike, no planned unit, and no worktree, and it never reads `design:` to invent a unit breakdown. The expected next step is `ws-focus` to name what you're aiming for; later, `ws-next` consults focus, the design, and follow-ups when routing.

```
$ /ws-init task templates
```

## `ws-start` `<ws-id> <what this unit does> [--base <unit-id|branch>] [--slug <slug>]`

**Use when** starting a new unit of work in an existing workstream — its own worktree plus ledger entry. Run `ws-init` first if no workstream exists.

Mints `unit-id = <ws-id>:<slug>`, resolves the repo (`--repo` › a `--base` unit's repo › cwd) and the base, then creates the `<slug>` branch and worktree off the base via the active `worktree-management` flavor's `create`. Appends the ledger line and seeds `charter.md` (the unit's static intent), `progress.md`, and `log.md`; `ws-resume` plans and builds. The `<slug>` is a shortened form of `<what>` — filler words dropped, four words and 32 characters at most — so branch names stay readable while the full sentence lives on in the ledger title and `charter.md`. Pass `--slug` to name it yourself.

```
$ /ws-start 2026-07-03-task-templates "add is_template flag" --base schema-migration
```

### `ws-spike`

`<ws-id> "<what to investigate>" [--slug <slug>] [--spawned-from <unit-id>] [--needs <target>]`

**Use when** starting research or an audit inside a workstream before building a unit — store-only, no worktree or PR. Requires `design:` on the workstream.

Creates a spike sibling to units: appends `spikes.md`, seeds `spikes/<slug>/` (`charter.md`, `progress.md`, `log.md`, `artifacts/`), and optionally seeds `## Needs` from `--needs`. Refuses slug collisions with units, spikes, or planned backlog lines — promotion to a build unit later uses a *distinct* slug via `ws-start`. No branch, no forge state. Expected next step: `ws-resume <spike-id>` (explicit id required — zero-arg resume cannot reach a spike).

```
$ /ws-spike 2026-07-03-task-templates "audit auth flows before implement-auth" --needs perf-baseline
```

### `ws-resume` `[unit-id|spike-id]`

**Use when** resuming or continuing an existing unit or spike, or picking work back up from inside its worktree (units only).

Dispatches by kind after resolving the bare slug against both ledgers. **Units:** with no argument, infers from the current worktree's branch; ensures the worktree exists, reconciles base drift, then plans/executes/ships via flavor ops. When `spec-driven-development=superpowers-prewalk`, unit phases run `plan` → `ws-prewalk` → hard stop → `plan-pause` → `loop` — see Prewalk. When `review=ws-critic`, completed units run a fresh read-only review between `loop` and `ship-pause` — see Critic. Pass `phase.py --skip-extension prewalk` or `--skip-extension critic` to bypass those phases once. `/ws-resume` must run in the parent session, not a subagent. **Spikes:** always need an explicit id; store-scoped research loop — no worktree, no ship phase — plan tasks, work in `artifacts/`, amend the umbrella design spec on the final task, then route via `ws-next` when spike-complete. Spikes skip prewalk and critic.

```
$ /ws-resume add-is-template-flag
$ /ws-resume audit-auth-flows
```

**`ws-board`** `[ws-id] [unit-id|spike-id]`

**Use when** you want to see or share where a workstream stands — "show the board", "what's done", "workstream status".

Reads both ledgers, derives each unit's and spike's status and task counts, and renders copy-paste-ready markdown grouped by state (Done, In progress, Blocked, Not started, Dropped) with open follow-ups listed across units. Spikes share the same kanban columns with an inline `[spike]` tag; the header reads `items done` when spikes exist. When an outcome is active in `focus.md`, a `Focus:` line appears above the table. Given a bare slug, prints that unit's or spike's full checklist and recent notes.

```
$ /ws-board 2026-07-03-task-templates
```

**Tip.** `ws-board` is strictly read-only — it derives everything and reconciles nothing. Reach for it freely to snapshot progress without touching git or GitHub.

**Blocked units and spikes.** A unit or spike is `blocked` whenever it has an unmet need — its base (units only), a spike, another unit, or a dependency added with `ws-block` — and shows in the board's ⛔ Blocked column until that need clears. Spike needs are satisfied at spike-complete (all tasks checked).

**`ws-block`** `<unit|spike> needs <target> ["note"] | <unit|spike> clear N<n>`

**Use when** a unit or spike must wait on another before it can proceed — record, view, or remove a dependency: "B needs A", "B is blocked by A".

Adds or removes **needs** on the dependent's `progress.md` `## Needs` section (under `units/<slug>/` or `spikes/<slug>/`). A `<target>` is a unit, a spike, or a follow-up (`<unit-id>:F<n>` / `WF<n>`); units satisfy at code-complete, spikes at spike-complete, follow-ups when checked or claimed. Any unmet need derives `blocked`. `clear N<n>` removes a need on a genuine scope change — it never marks one satisfied, since satisfaction is always derived.

```
$ /ws-block add-is-template-flag needs schema-migration
```

## ws-backlog

```
"<what>" [--defer | --here | --plan --base <x> [--needs a,b]] [--to <unit>]
```

**Use when** future work surfaces mid-workstream and needs capturing — a bug or follow-up out of scope for the unit you're in, a "we should also do X later" idea, or a whole unit worth planning.

Routes the item to its one correct home so nothing orphans: a unit's `progress.md` `## Follow-ups` (`F<n>`, fixed before this unit's PR merges), the workstream's `backlog.md` `## Follow-ups` (`WF<n>`, deferred), or `backlog.md` `## Planned units` (a dependency reservation for a future unit — records `base=` / `needs=`, not a roadmap slice `ws-next` will auto-start). Inside a unit it asks the one thing it can't derive — resolve before this PR merges? — while flags decide it non-interactively. It never writes `## Tasks`: the plan comes from planning, not by hand.

```
$ /ws-backlog "retry backoff is wrong" --defer
```

## ws-focus

```
list | add "<outcome>" | activate <n|slug> | done [n|slug] | move <from> <to> [ws-id]
```

**Use when** you want to name what the workstream is aiming for next — a demo milestone, a user-visible outcome, or a north-star slice — and steer what `ws-next` proposes.

Maintains `focus.md` — a manual outcome queue, separate from units and backlog. Open focuses keep insertion order; one line is **active** (`[>]`) at a time; the rest are queued (`[ ]`) or recently done (`[x]`, last three kept). Nothing auto-advances — you activate explicitly. `add` appends as queued; `list` shows numbered open items; `activate` switches which outcome is active (by number or slug); `done` marks one complete; `move` reorders the open list. Workstream-scoped, store-only.

```
$ /ws-focus add "user can see a live board"
$ /ws-focus list
$ /ws-focus activate 1
$ /ws-focus done
```

## ws-restack `<unit-id> <new-base>`

**Use when** a unit's base PR merged and its branch must move onto a new base, or GitHub auto-retargeted a dependent PR and the local branch needs realigning.

Resolves `<new-base>` (a branch, or another unit-id → that unit's branch) and rebases per the SPEC reconciliation. When *we* initiate — the PR's `baseRefName` is unchanged remotely — it runs `gh pr edit --base` first; in the auto case it skips that. Stops and reports on conflicts.

```
$ /ws-restack add-is-template-flag master
```

### `ws-drop` `<unit-id|spike-id>`

**Use when** you want to abandon a unit's worktree and branch, or abandon a spike's research — not when work already shipped or the spike is complete.

**Units:** shows exactly what will be removed and requires explicit confirmation, then removes the worktree and deletes the *local* branch. Appends a `dropped` log line but keeps `progress.md` and the ledger line as the reconciliation record. **Spikes:** store-only — no worktree teardown; refuses drop when the spike is already complete (mirror merged-unit guard). Dropping never releases blocked dependents — they show `(dropped)` until re-pointed.

```
$ /ws-drop add-is-template-flag
```

> **Gotcha.** `ws-drop` never deletes the remote branch or closes the PR unless you ask, and it preserves the store. To restart, run `ws-start` with the same intent — it versions the id with a `-N` suffix and records `restart-of` automatically.

### `ws-next` `[ws-id]`

**Use when** you're unsure which `ws-*` command or which unit to act on next — after finishing a unit, when a PR merges, or any "what now?" moment.

Reads both ledgers and each unit's and spike's live status, then lists every unit or spike you could act on right now — ranked, with one marked as the default — and what each needs. Spike moves use `ws-resume <spike-id>` with no `branch=` tail. When nothing can move, it walks you through proposing the next unit: pick a lane (follow-ups, design spec, or active focus), then pick from up to three composed candidates. Terminal spikes appear in `Covered:` so proposals skip colliding slugs. It decides, it doesn't do the work (`ws-resume` does). Read-only, like `ws-board`.

```
$ /ws-next 2026-07-03-task-templates
```

## `ws-config`

```
[show | set <group> <flavor> | set-overrides <path> | list [group] | add <group> <flavor>]
```

**Use when** viewing or changing workstream flavors — which external tool backs each behavior group.

Edits only `~/.local/share/workstreams/flavors.ini` and the spec-watch script under the store's `hooks/` — never a worktree. `show` (the bare default) prints the active flavor per group and which layers are in play, annotates which flavors' tools are detected, flags an active flavor whose tool is missing, and offers to activate detected flavors for unset groups; `set` switches one group's flavor; `add` scaffolds a custom flavor stub; `set-overrides` points at an external overrides file. Every run also reconciles the spec-watch script to the active flavors. See the Flavors section.

```
$ /ws-config
$ /ws-config set worktree-management wmx
```

# Focus, backlog & follow-ups

Three store homes capture different kinds of "not done yet" — an outcome you steer toward, a future unit you've reserved, or a follow-up to fix later. They don't overlap: each has its own file, lifecycle, and role in routing.

| What | Where | Written by | Role |
|---|---|---|---|
| **Focus outcome** | `focus.md` `## Focus` | `ws-focus` | Steers `ws-next` proposals — "what are we aiming for?" Not a unit; no branch or PR. |
| **Planned unit** | `backlog.md` `## Planned units` | `ws-backlog --plan` | Dependency reservation for a future `ws-start` — records `base=` / `needs=`. `ws-next` does *not* auto-start these; `ws-board` shows them as not started. |
| **Follow-up** | Unit `progress.md` `F<n>` or backlog `WF<n>` | `ws-backlog` | Captured work — a bug, polish item, or "do later." In-unit `F<n>` closes before the PR merges; deferred `WF<n>` outlives the unit. |

**Focus** is steering, not execution. One outcome is active (`[>]`) at a time; others queue (`[ ]`) in insertion order or sit in recent history (`[x]`, last three kept). When set, `ws-next` prefers design-sourced candidates that advance the active focus over non-blocking follow-ups, and `ws-board` prints a `Focus:` line above the status table. Nothing promotes or completes a focus line on its own — use `ws-focus activate` and `ws-focus done`.

**Planned units** are not a roadmap queue. They record stack and dependency fields so a later `ws-start` can pick them up with the right `base=` and `needs=`. A line disappears from "not started" once a ledger slug matches — no manual check-off.

**Follow-ups** are concrete items, not north-star outcomes. Route in-unit when you'll fix it before merge; defer to the workstream backlog when it can wait. A follow-up unit (`claims=` on its ledger line) closes a batch without re-proposing them.

> **When to use which.** Name the milestone with `ws-focus add`. Reserve a future unit's dependencies with `ws-backlog --plan`. Capture a specific bug or polish item with `ws-backlog --defer` or `--here`.

# Flavors

External tools are pluggable. The skills never hardwire a worktree tool, a planning methodology, or a forge — each is a **flavor** you choose, so the same `ws-*` commands drive whatever you've configured.

A **group** is a fixed behavior category the skills define; a **flavor** is one implementation of it; an **operation** is a named slot a flavor fills with a one-line instruction (a shell command, or a skill to invoke). Exactly one flavor per group is active, globally. Swapping a flavor changes only mechanism — the ws bookkeeping (progress, log, ledger, PR-ready) is intrinsic and never moves.

| Group | Operations | Built-in flavors (default first) |
|---|---|---|
| `worktree-management` | create · remove · locate | `git-worktree`, `wmx` |
| `spec-driven-development` | plan · execute · ship · spec-glob | `none`, `superpowers` |
| `forge` | default-branch · pr-status · pr-create · pr-ready · pr-retarget | `gh` |

Your selection and any custom flavors live in `~/.local/share/workstreams/flavors.ini`, merged over the plugin's built-in defs; an optional overrides file layers on top. View or change all of it with `ws-config`.

> **Configure with `/ws-config`.** Bare `/ws-config` shows the active flavor per group, detects installed tools, and offers to activate matching flavors for groups you haven't set; `ws-config set <group> <flavor>` switches one; `ws-config add` scaffolds a custom flavor to fill in. `gh` is the assumed forge baseline — a non-GitHub user adds a custom `forge` flavor via the overrides file.

## Customizing

Flavor definitions merge across three layers, lowest to highest precedence — the plugin's built-in `flavors.ini`, your store `~/.local/share/workstreams/flavors.ini`, then an optional external overrides file. Higher layers win **per operation**, so you redefine only the keys you care about and inherit the rest.

1. **Switch the active flavor** — the common case. `ws-config set <group> <flavor>` rewrites the group's line under `[active]` in your store file; nothing else changes.
2. **Customize or add a flavor** — `ws-config add <group> <flavor>` scaffolds a `[<group>/<flavor>]` section to fill in. Redefine only the operations you want to change.

3. **Layer an external file** — `ws-config set-overrides <path>` records `overrides-file=<path>` under `[config]`, so a file outside the store (a shared team config, say) wins over both lower layers.

Each operation's instruction is a `plugin:skill` id (invoked as a skill), a `ws-*` command line (invoked as that skill), or a shell command — with `<branch>`, `<base>`, `<path>`, `<repo>`, `<pr>`, and `<new-base>` filled from context; hook instructions and their prompts/choices may also use `<unit>` and `<command>` (the firing skill's resolved next command). A non-GitHub user, for instance, replaces the `forge` group with a custom flavor:

```
; ~/.local/share/workstreams/flavors.ini
[active]
forge = glab

[forge/glab]
default-branch = glab repo view -F json | jq -r .default_branch
pr-status      = glab mr view <branch> -F json
pr-create      = glab mr create --source-branch <branch> --target-branch <base> --draft
pr-ready       = glab mr update <pr> --ready
pr-retarget    = glab mr update <pr> --target-branch <new-base>
```

> **Missing keys fall back.** An operation no layer defines resolves to the group's **default** flavor ( `git-worktree` / `none` / `gh` ); an unreadable `overrides-file` is warned and skipped. Define every operation your custom flavor needs — don't lean on a partial section.

## Flavor hooks

A **hook** lets a flavor react at a skill's lifecycle point — an optional operation named `hook-<skill>-<event>`. Each skill documents the events it fires; at an event it runs that hook from every active flavor that defines it. Hooks fire **only in an interactive session** — a subagent or headless run fires none, so a hook never steals focus or blocks automation.

Hooks can prompt. Companion keys (valid only on `hook-*` operations) make one interactive: with no `.prompt` the hook just runs; a `.prompt` and no choices is a yes/no (Yes runs the base instruction, No skips); adding `.choices.<name>` entries turns it into a 2–4-option pick, each with an optional `.choices.<name>.desc` (the chosen option's instruction runs, the base is ignored; an empty instruction — and a dismissed prompt — skips).

A hook at a skill's next-step offer is a **chain hook**: its prompt replaces the default opt-out offer (proceed unless declined). The skill stays unaware of what the choices mean — a choice resolving to `<command>` runs the named command in this session; any other choice is the flavor handing the work off its own way, so the skill runs it, re-emits the command, and stops.

The built-in `wmx` flavor uses both kinds: it creates worktrees windowless, then at the end of `ws-start` asks whether to open a new window, and at the end of `ws-next` asks where the picked unit should run.

```
; built-in — worktree-management/wmx
create                              = wmx worktree create <branch> --base <base> --no-window
hook-ws-start-after                 = wmx window open <branch>
hook-ws-start-after.prompt          = Open <branch> in a new window? (No = keep working
here)
hook-ws-next-after.prompt                = Where should <unit> run?
hook-ws-next-after.choices.skip          =
hook-ws-next-after.choices.skip.desc     = Not now
hook-ws-next-after.choices.here          = <command>
hook-ws-next-after.choices.here.desc     = Here
hook-ws-next-after.choices.window        = wmx window open <branch>
hook-ws-next-after.choices.window.desc = New window
```

Choices appear in the order the flavor file lists them, and the first is the safe one — a dismissal never starts work. An option whose instruction names something the skill can't supply is dropped from the pick: on a `start` move the unit has no branch yet, so "New window" disappears and the pick narrows to not-now-or-here — `ws-start` offers the window itself once the worktree exists.

> **Silence or force it to taste.** In your store `flavors.ini`, set `hook-ws-start-after.prompt =` (empty) to always open without asking, or `hook-ws-start-after =` (empty) to never open — one line each, merged over the built-in. The same levers work on `hook-ws-next-after` and its `.choices.*` keys.

## Spec-watch: from spec to workstream

Workstreams usually begin at a design spec — and the moment one is written is easy to miss. **Spec-watch** closes that gap: when a written file matches the active `spec-driven-development` flavor's `spec-glob` and no workstream's `design:` claims it, the runtime nudges the session to offer `ws-init` with that spec as the design. A workstream that has no design yet is surfaced as the attach-instead alternative. It only suggests — single-PR work is left alone, and nothing is created without you.

The mechanism costs nothing when idle: the plugin's file-write hook merely checks whether `~/.local/share/workstreams/hooks/spec-watch-<flavor>.sh` exists and runs it — the installed script *is* the on-switch. `ws-config` keeps it in sync on every run: a flavor that declares `spec-glob` gets the script installed with its glob baked in; a flavor without one gets it removed. Ownership is matched by the spec's **filename** against `design:` lines, so `~`, absolute, and symlinked spellings all count.

> **Opt out in one line.** Set `spec-glob =` (empty) on your active flavor in the store `flavors.ini`, then any `/ws-config` run removes the watcher — the same empty-key convention that silences flavor hooks.

## Using with Superpowers

The `superpowers` flavor of `spec-driven-development` wires the ws commands into the Superpowers loop — but it moves where planning happens. On its own, Superpowers runs **brainstorm** → **write-plan** →

**execute** in one branch. With workstreams the top-level plan step drops out for multi-unit work: a feature is many units, and each unit plans itself. Single-unit work can use `ws-oneshot` after spec-watch.

1. **Brainstorm the spec.** Run the Superpowers `brainstorming` skill as usual to produce the design doc.
2. **Pick an entry path at spec time.** Accept the spec-watch nudge and run `ws-init` — then `ws-focus` and split with `ws-start`. Single-unit scope may use `ws-oneshot` (init + start + resume in one confirmed chain) instead of focus + manual start.
3. **Multi-unit: split the spec into units** with `ws-start`, one slice at a time — each unit's `charter.md` captures that slice's intent against the shared design.
4. **Let each unit plan itself.** First `ws-resume` on an unplanned unit runs `writing-plans` (plan saved as `<design-dir>/<slug>-plan.md`).
5. **4a (optional). Prewalk before execute.** When `spec-driven-development=superpowers-prewalk`, the next `ws-resume` runs read-only codebase exploration, writes `prewalk.md`, then stops for a model switch before cheaper execution. See Prewalk.
6. **Confirm plan at plan-pause.** At **plan-pause** pick Subagent-driven or Inline from the flavor execute picker. On confirmation, `confirm_plan.py` derives tasks into `progress.md` and appends the `plan=done` receipt. Later runs print orientation (goal, progress, plan path) then continue executing.

> **Planning lives in units.** After the spec, go `ws-init` → `ws-focus` → `ws-start`. `writing-plans` is deferred into each unit's `ws-resume`. **Oneshot:** `ws-init` → `ws-start` → `ws-resume` — one workstream, one unit, one confirmed chain.

> **Plan path.** Unit plans live at `<design-dir>/<slug>-plan.md` next to the workstream design spec — not in `docs/superpowers/plans/`. The store records the path in `log.md` as `plan <absolute-path>`.

## Prewalk (superpowers-prewalk)

The `superpowers-prewalk` flavor extends `superpowers` with optional prewalk. It complements Superpowers — it does not replace `writing-plans`. After a unit plan is written, `ws-resume` invokes the `ws-prewalk` skill for read-only codebase exploration; the parent session writes store artifacts and log lines. No source edits happen during prewalk.

**Why use it. Cost reduction:** front-load exploration on a frontier model, then switch to `cheap-model` for plan-pause and the implementation loop — the expensive model covers planning and read-only discovery once; bulk execution runs cheaper. **Better execution context:** `prewalk.md` gives the cheap model file paths, patterns, and constraints up front, so loop turns spend fewer tokens re-exploring.

**When to enable.** Large or unfamiliar codepaths where upfront exploration pays off and you want most turns on a cheaper model. Activate with `ws-config set spec-driven-development superpowers-

`prewalk`, pin `agent`, and set `cheap-model.<agent>` via `set-config` (required). Optionally set `frontier-model.<agent>` for your planning model — no bundled defaults. Run `/ws-config` to see `required:` and `recommended:` lines until config is complete. Skip on spikes, in headless runs, or for grandfathered plans (a plan written before prewalk was activated).

**Unit resume flow** when prewalk is enabled:

`plan` → prewalk-config → `prewalk` → hard stop → `plan-pause` → `loop`

`prewalk-config` runs only when `agent` or `cheap-model.<agent>` is unset — `ws-resume` stops and prints `required:` lines from `ws-config show`; no exploration until you `set-config` and re-run.

```
$ /ws-config set spec-driven-development superpowers-prewalk
$ /ws-config set-config agent cursor
$ /ws-config set-config cheap-model.cursor <your-cheap-model>
$ /ws-config set-config frontier-model.cursor <your-frontier-model>
```

**Model handoff.** After prewalk completes, `ws-resume` stops. Run `/model <cheap-model>`, then `/ws-resume <unit>` again. Plan-pause step 0 is the model-switch gate — not a separate durable phase.

**Artifacts and log.** `units/<slug>/prewalk.md` holds the exploration digest. The log records `decision prewalk=done plan=<path> digest=<8-hex>` — not prose.

> **Superpowers still plans.** `writing-plans` runs in the plan phase; prewalk runs only on the unit path, between plan completion and plan-pause confirmation. See ws-resume.

▶ **Prewalk FAQ**

## Critic (review/ws-critic)

The `ws-critic` review flavor adds a fresh-context, read-only adversarial review after every unit is code-complete and before `ship-pause`. It checks the charter, design, implementation plan, complete branch diff, surrounding code, and tests.

**On by default.** Fresh installs use `review=ws-critic`. Disable with `/ws-config set review off`. The setting is independent of the spec-driven-development flavor.

**Unit resume flow** when critic is enabled:

`loop` → `critic` → `ship-pause` → `draft-pr` → `done`

**Advisory by design.** The reviewer reports Critical, Important, and Minor findings plus a verdict. A `NOT READY` verdict is shown at `ship-pause`; it does not remove the user's ability to fix, skip, or ship.

**Artifacts and reruns.** `units/<slug>/critic.md` stores the report. The log binds the result to the current diff with `decision critic=done verdict=... digest=<8-hex>`. New commits cause a fresh review.

**Skip paths.** Use `phase.py --skip-extension critic` for one resume. Headless runs and units grandfathered before activation skip it. Critic applies to units, not store-only spikes.

▶ **Critic FAQ**

# Cheat sheet

The workflow commands, one durable store: worktrees hold code, the store holds truth, and status is always derived live from git and GitHub.

| Command | Purpose |
|---|---|
| `ws-init` | Create a new workstream container (metadata + empty unit and spike ledgers + empty backlog + empty focus). Run before any unit or spike exists. Expected next: `ws-focus`. |
| `ws-oneshot` | Single-unit entry: `ws-init` → `ws-start` → `ws-resume` in one user-confirmed chain. Offer when scope reads one PR; never auto-run. |
| `ws-start` | Start a unit: its own worktree, branch, ledger line, and `progress.md` / `log.md`. The only thing that creates a unit. Refuses spike-reserved slugs. |
| `ws-spike` | Start store-only research: spike ledger line, `spikes/<slug>/`, no worktree. Requires `design:`. Ends by amending the umbrella spec. Chain: `ws-resume <spike-id>`. |
| `ws-resume` | Advance a unit or spike at any stage: plan, optional prewalk, execute loop, optional critic, ship (units) or spec amend (spikes) — reopening a gone worktree and reconciling base drift as needed for units. |
| `ws-board` | Show or share where a workstream stands. Units and spikes on one kanban; spikes tagged `[spike]`. Read-only. |
| `ws-block` | Add or remove needs on a unit or spike. Targets may be units, spikes, or follow-ups. An unmet need derives `blocked`. |
| `ws-backlog` | Capture future work — routes a follow-up (`F<n>` / `WF<n>`) or a planned unit (dependency reservation) to its right home so nothing orphans. |
| `ws-focus` | Maintain the outcome queue in `focus.md` — set, switch, or complete what the workstream is aiming for; steers `ws-next` proposals. |
| `ws-next` | List every unit or spike that can move right now, ranked, then ask which one moves and where it runs (units only). When nothing can move — or alongside in-flight work — propose the next unit by lane. Read-only. |
| `ws-restack` | Move a unit's branch onto a new base after the base PR merged or GitHub auto-retargeted. |
| `ws-drop` | Abandon a unit (worktree + local branch) or a spike (store log only). Refuses complete spikes and merged units. Keeps the reconciliation record. |

| Command | Purpose |
| --- | --- |
| `ws-config` | View or switch flavors — the external tool backing each behavior group. Edits `flavors.ini` only, and reconciles the spec-watch script. |

```
~/.local/share/workstreams/<ws-id>/
├── workstream.md        # metadata only (id, name, goal, design)
├── units.md             # append-only ledger: unit ↔ repo/branch identity
├── spikes.md            # append-only ledger: spike identity (no branch=)
├── backlog.md           # planned units + deferred follow-ups
├── focus.md             # outcome queue (steers ws-next)
├── units/
│   └── <unit-id>/
│       ├── charter.md      # STATIC: why this unit exists (the north star)
│       ├── progress.md     # MUTABLE work-state: Tasks + Follow-ups checklists
│       ├── log.md          # APPEND-ONLY: created · dropped · restack · decision · note
│       └── prewalk.md      # optional: exploration digest (superpowers-prewalk)
└── spikes/
    └── <slug>/
        ├── charter.md      # STATIC: what to investigate
        ├── progress.md     # MUTABLE: Tasks + Needs only
        ├── log.md          # APPEND-ONLY: created · dropped · decision · plan
        └── artifacts/      # working drafts, spec-before snapshots
```

`ws-id` = `<YYYY-MM-DD>-<slug(name)>` · `unit-id` / `spike-id` = `<ws-id>:<slug>` (bare `<slug>` resolves against both ledgers; `-N` = restart) · `branch` = `<slug>` (units only). The store is global, not per repo.

> **Worktree = code only.** Never write store files into a worktree. Find a unit's worktree via its ledger branch; drop and recreate worktrees freely — progress survives in the store.

> **progress.md = work-state source of truth (mutable).** Tasks and follow-ups live here and nowhere else. Ids ( `T1` , `F1` ) are monotonic per unit and never reused, even after check-off or removal. No history is kept here.

> **log.md = append-only.** One line per event — `created`, `dropped`, `restack`, `decision`, `note`. Never edit or delete prior lines; the log never stores current state.

> **Nothing volatile is stored — derive it live.** Current branch comes from git, base and retargeting from `gh pr view --json baseRefName`, PR number and draft/ready/merged from GitHub, and status from a first-match cascade: dropped → merged → blocked → in-review → building.

# Gotchas & FAQ

Worktrees are disposable code checkouts; every durable fact lives in the global store or is derived live from git and GitHub — so most "I lost it" moments are recoverable.

> **Golden rule.** Never store what git or GitHub owns. Branch, base, PR number, and status are always derived live; only tasks, follow-ups, and history are written to the store. If a worktree feels lost, the store still knows the unit.

| Question | Answer |
|---|---|
| **Q:** When do I use a spike instead of a unit? | **A:** When the work is research or an audit that should amend the design spec but may never ship as a PR — audit before build, investigate an open question, resolve unknowns mid-flight. Run `/ws-spike`, then `/ws-resume <spike-id>`. When findings need product code, start a separate unit with a *new* slug via `ws-start`. |
| **Q:** Can I resume a spike with no argument? | **A:** No. Zero-arg `ws-resume` infers from the cwd branch and only scans unit ledgers. Spikes have no branch — always pass the spike id explicitly. |
| **Q:** I deleted the worktree — is my work lost? | **A:** No. Worktrees are disposable; all durable state lives in `~/.local/share/workstreams/<ws-id>/`. Run `/ws-resume <unit-id>`: if the branch still exists it reopens the worktree; if the branch is also gone it starts fresh off the repo default, using the store's `progress.md` as the restart baseline. |
| **Q:** Which unit am I in? | **A:** Run `/ws-resume` with no argument. It infers the unit from the current worktree's branch by scanning every `units.md` ledger in the store, then resolves the `ws-id`, repo, and branch for you. |
| **Q:** My base PR merged — how do I move onto the new base? | **A:** Run `/ws-restack <unit-id> <new-base>`. It rebases `--onto` the new base from the recorded merge-base and appends a `restack` line to `log.md`. When you initiate and the PR base is unchanged remotely, it also runs `gh pr edit --base` first. |
| **Q:** GitHub auto-retargeted my dependent PR — do I still restack? | **A:** The realign still happens, but the `gh pr edit` step is skipped since GitHub already moved the base. `/ws-resume` also auto-reconciles: if the live `baseRefName` differs from the recorded base, it realigns and records a `restack` line before continuing. |
| **Q:** I opened a per-unit window and it didn't auto-start the agent. | **A:** Windows are optional — run `ws-resume` in your current session, or launch `claude` in the window yourself. A window only autostarts the agent when your `worktree-management` flavor is configured to. |

| Question | Answer |
| --- | --- |
| **Q:** What is prewalk and when should I enable it? | **A:** A `superpowers-prewalk` flavor (`extends = superpowers`) that runs read-only exploration after a unit plan — frontier model discovers context once, then a cheaper model runs the loop. Enable with `/ws-config set spec-driven-development superpowers-prewalk`, pin `agent` via `set-config`, set `cheap-model.<agent>` (required), and optionally `frontier-model.<agent>` (recommended). See Prewalk. Spikes skip it. |
| **Q:** Why does ws-resume stop after prewalk? | **A:** Prewalk ends with a hard stop so you can `/model <cheap-model>` before plan-pause and the implementation loop — separating expensive discovery from cheaper execution. |
| **Q:** Can a workstream span multiple repos? | **A:** Yes. The store is global, not per-repo. Each unit records its own `repo=<org/repo>` (and branch) on its ledger line in `units.md`, so units of one workstream can live in different repositories. |

**Optional window.** A per-unit window is optional — run `ws-resume` in your current session instead. If you do open one and it doesn't start the agent, run `claude` manually (a window only autostarts when the `worktree-management` flavor is configured to).

**Tip.** Read-only by design: `ws-board` never reconciles a base. Only `ws-restack` (explicit) and `ws-resume` (on detecting drift) move a branch onto a new base.